BUILD RECORD · HACKATHON

Everything that broke, and *what it taught us*

Context Switch is an AI communication translator: say what you mean, understand what someone else meant, and work out what an argument is actually about. This is the engineering record — the architecture, the eighteen logged failures, and what verification actually proved.

STACK VITE · REACT 18 · TYPESCRIPT STRICT · ZOD **DURATION** ~12.5 HOURS

RUNTIME DEPS 4

17,239 LINES TS / TSX / CSS	113 AUTOMATED GATE CHECKS	18 LOGGED FAILURES	20 RECORDED DECISIONS
9 CONTRAST-AUDITED THEMES	0 CONTRAST FAILURES / 443 NODES		

01

What it is

Three features, each one a different direction through the same problem.

Communicate is the translator. You pick who you are and who you are talking to, then write what you actually mean; it returns a version they can hear, plus a plain account of what changed and why. It runs in reverse too — paste or upload something you received and it separates what the message says from what you might be adding to it.

Repair takes a screenshot of an argument, or a description of one, and maps it: each side on its own terms, where the temperature rose, and what the disagreement is actually about underneath the surface topic. It never decides who is right.

Inspect is introspection by clickable question — 68 nodes, 34 questions, seven branches, each next question chosen by the previous answer, ending either in something that helps you feel better or something you can actually say out loud.

The product rule underneath all three: **never claim to know what someone meant.** Every inference the model makes is rendered with its support level — strongly supported, plausible, speculative — and its evidence from the text. A read that claims certainty it does not have is a product failure, and the fixture gate fails the build over it.

CONTENTS

What is in this record

01	What it is	2
	Decisions and agent communication	4
02	Four constraints that shaped everything	7
03	How it was built	9
04	The failure ledger	10
05	What verification actually proved	16
06	The last mile	17
07	Known limits	18

DECISIONS

Decisions and agent communication

The build ran as one orchestrator directing managed subagents. Two things are worth the record: what got decided, and how the agents actually talked to each other — because several of the most useful findings arrived through the conversation rather than the code.

Twenty recorded decisions

ID	DECISION	WHY IT MATTERED / WHAT IT COST
D-001	Vite + React 18 + TypeScript with **no server**; live AI reaches the provider through a Vite dev-server middleware plugin that holds the key in the Node process.	Bought a fast local demo. Cost: a statically hosted build has no Node process, so live AI there needs a user-pasted key. The most consequential tradeoff in the build.
D-002	Pin Tailwind CSS v3.4 rather than v4.	Three subagents were about to write Tailwind classes in parallel. Reducing variance mattered more than being current.
D-003	Add `tsx` as a dev dependency for the fixture-validation script instead of a test runner.	Dev-only tooling that never enters the bundle; Vitest was a much larger addition for one assertion file.
D-004	Make the Say It Better Zod schema **stricter than the spec type**: when `needsFollowUp` is false, `sendableMessage` is required.	Every result field was optional in the spec, so an empty result validated and rendered a blank screen. Closed at the gate rather than in five consumers.
D-005	Require exactly two conflict `participants`, joined to speaker names on `speakerId`.	The response carries no names, so a mismatch rendered blank columns. Enforcing the join turned a silent blank into a caught error.
D-006	Parallelise across managed subagents with **disjoint file ownership** and frozen API contracts.	Made a five-phase build fit the window. Worked only because contracts, schemas and tokens were written centrally first.

ID	DECISION	WHY IT MATTERED / WHAT IT COST
D-007	Require Decode's `unknowns`, `knownFacts` and purpose lists to be non-empty, and exactly three response options.	The known / inferred / unknown split *is* the mode. A decode claiming nothing is unknowable has failed the product rule, so the gate rejects it.
D-008	Offer the three-way jargon control in the UI but map it onto the contract's boolean.	Honest cost: "remove" and "reduce" behave identically at the prompt level. Flagged rather than hidden.
D-009	When fixtures are the only client and the text is the user's own, **refuse with an explanation** instead of returning a prepared fixture.	Handing back the engineer's status update as a translation of someone's own paragraph is the difference between a demo and a fake.
D-010	Default the model to the mid-tier rather than the most capable one.	A three-minute demo is latency-sensitive; every extra second is dead air. Overridable with one line in `.env`.
D-011	Cascade to the next client only on `no_client_available` / `network`.	A timeout or schema failure is reported as itself rather than silently re-run down a slower path, doubling latency on the likeliest failure.
D-012	Rebuild the visual foundation on an editorial display serif with a warm textured ground and layered depth.	Turned a competent-looking prototype into something that reads as a product.
D-013	Remove all self-referential "demo" vocabulary from the UI while keeping every internal identifier unchanged.	Users should not read the scaffolding. Renaming the code too would have churned fixtures and state keys for no user-visible gain.
D-014	Pin the visual style and make style a **swappable layer** — every colour, radius, shadow and gradient moved into CSS custom properties.	Theme switching became a runtime token swap rather than a rebuild. The trap: colours must be stored as `R G B` triplets or every opacity modifier silently breaks.
D-015	Multi-provider relay with automatic failover across an ordered chain.	Paid for itself: the primary account ran out of credit mid-build and the app kept working on the next provider without a code change.
D-016	**Extend** the existing data contracts with the newer spec's fields rather than replacing them.	Kept the validation gate, every fixture and the whole AI layer intact while adding evidence and safety structures.

ID	DECISION	WHY IT MATTERED / WHAT IT COST
D-017	Add the dark palette as a ninth theme and make it the default, rather than replacing the theming system.	One brief's palette became one option among nine instead of a rewrite.
D-018	Skip React Router and Framer Motion despite both being suggested.	Routes were suggested, not required, and overlays fit the flows better. Two dependencies and a navigation model avoided.
D-019	Collapse Repair to three screens: put it in → check it → read it.	The mode was the most visually ambitious and the least legible. Fewer screens made it demonstrable.
D-020	Put the second provider first in the chain and prepare a bound fallback analysis for the demo conversation.	The one demo path behaves identically whether the AI is up or completely down.

How the agents were made to talk

The orchestrator wrote every shared shape first — contracts, Zod schemas, design tokens, the session reducer — and only then fanned out. Each agent got an explicit file allowlist, and the primitive API signatures were frozen in the brief so consumers and components could be written at the same time instead of one after the other. Across roughly five thousand lines of parallel work the total integration cost was a single type error.

The instruction that did the most work was this: **report a contract deviation, do not silently change the signature.** Agents complied and it kept catching things. One found that a safety notice had to be able to render nothing at all, so its return type had to widen. One found a colour token with no owner for a sixth safety category. Each came back as a question rather than a quiet edit that would have surfaced days later as a mystery.

> An agent that reports "accessibility verified" is worth less than one that reports the measured contrast ratio and the number of nodes it measured.

Findings that came from the conversation, not the code

- **The agent writing the README found two live defects.** A documented mode switch controlled nothing, because one variable was read server-side and a differently-named one client-side. And the flagship demo silently skipped its own scripted beat. Neither was reachable by the typechecker, the linter or the build — all green throughout. Documenting a system is a test of it.
- **An agent driving the app with a mouse** found that scrolling the page over a select in a sticky header swapped the loaded scenario and reset everything. Invisible to every automated check, and precisely the kind of thing that goes wrong on stage.
- **Three agents independently reported the same white card.** Two patched their own instance and moved on. Only collecting the three reports revealed one cause: class conflicts fell through to CSS source order, and colour utilities are emitted alphabetically, so the winner depended on the colour's name.
- **An agent corrected its own instrument.** Told to grep the bundle for a leaked key, it noticed the check would pass trivially because the code under test was not in the bundle yet, and rebuilt with a decoy value to make the test mean something.

Direct messages, mid-run

Coordination was not fire-and-forget. When a brand swap turned out to have created a second logo render site — so edits to the first had no visible effect — the orchestrator messaged that agent mid-run to fix the duplication at its cause rather than racing it for the same file. When a review found the flashcard exercise taught classification but never the reframe, the deck agent was sent back with a specific brief rather than rebuilt.

What went wrong with the agents themselves

Three died mid-run: one machine sleep, two stream stalls. Their file edits had already completed, so recovery was cheap — but each had left claims in the log that were never verified, and those were re-verified by hand rather than trusted. A dead agent's unfinished report is the most dangerous artefact in a parallel build, because it looks exactly like a finished one.

02

Four constraints that shaped everything

Almost every interesting decision descends from one of these.

NO SERVER, BUT NO KEY IN THE BROWSER

A hackathon build has no backend, and an API key in a `VITE_*` variable is inlined into the bundle. The answer was a **Vite dev-server middleware plugin** — 525 lines of Node running inside the dev server, reading the key via `loadEnv`, never exposing it to the client.

Proven, not assumed: a decoy key of the real `sk-ant-…` shape was written to `.env`, a production build run, and `dist/` grepped for the decoy *value*. No matches.

ONE VALIDATION GATE, NO EXCEPTIONS

Every model response — live, or from a fixture — passes the same `validateResponse(mode, candidate)` Zod gate. On failure: retry once with a repair instruction, then a typed error. **Raw model output is never rendered.**

The schemas are strict on purpose. Decode's "what you cannot know" list is *required non-empty*, so a response claiming nothing is unknowable is rejected rather than displayed.

STYLE AS A RUNTIME LAYER

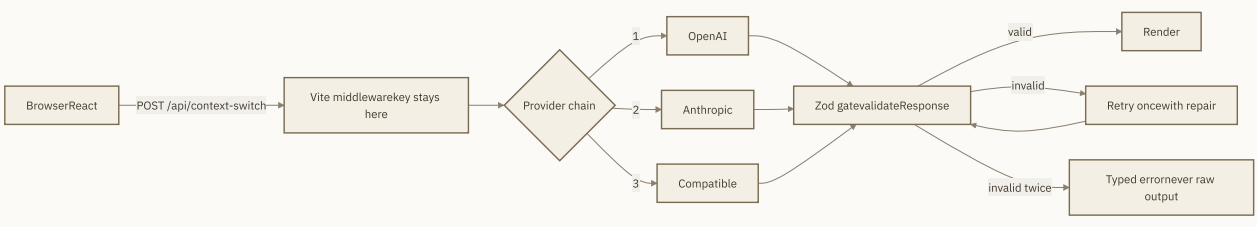
The visual direction was deliberately deferred, so every colour, radius, border width, shadow and gradient moved out of the Tailwind config into CSS custom properties. Tailwind maps names onto variables; switching themes is a runtime token swap, not a rebuild.

The trap: colours must be stored as `R G B` triplets, because Tailwind wraps them as `rgb(var(--token) / <alpha-value>)`. A hex value there silently breaks all **~165** opacity modifiers in the codebase.

A PROVIDER CHAIN, NOT A PROVIDER

`anthropic → openai → compatible`, configured in `.env`, with automatic failover. Cascade fires only on `no_client_available` and `network` — a timeout or a schema failure is reported as itself, because re-running every failure down the chain doubles latency on exactly the failure a live demo is most likely to hit.

This turned out to matter. The Anthropic account ran out of credit mid-build; the app kept working on OpenAI without a code change.



The relay is the only place the key exists. Everything the user sees has passed the gate.

03

How it was built

Managed subagents with disjoint file ownership and frozen API contracts. The orchestrator owned every shared shape — contracts, schemas, tokens, the app shell — so parallel agents could not diverge on them.

That worked. Three agents built the three result views simultaneously because those views share no files, only the already-frozen contracts and primitives. Agents reported contract *deviations* rather than silently changing signatures, and one negative-tested its own gate by injecting a defect and confirming the build went red.

It also had limits worth recording. Three agents died mid-run — one machine sleep, two stream stalls. Their file edits had completed, so recovery was cheap, but each had left claims in the log that were never verified. Those were re-verified by hand rather than trusted.

PHASE	WHAT LANDED	GATE AT THE END
0–1	Recon of three source documents, seven spec ambiguities raised before any code. Contracts, Zod gate, design tokens, 13 UI primitives, vocabulary, six fixtures.	`tsc 0 · eslint 0 · fixtures 43/43`
2–4	The AI layer without a server: proxy plugin, router, retry-and-repair. All three result views in parallel.	`+ functional 53/53`
5	Visual foundation, then the whole style layer extracted into runtime tokens. Nine themes.	`contrast 0 / 443 nodes`
6–7	Rehearsals, accessibility measurement, provider failover proven end to end against a local mock.	`live pipeline verified 25/25`
8	A twelve-feature "observability console" rebuild, per brief. **Rejected outright** — see F-016.	`gates green, product wrong`

PHASE	WHAT LANDED	GATE AT THE END
9	Rebuilt as three plainly-named features. Repair collapsed to three screens, progressive first read, bound fallback.	`fixtures 60/60 · functional 53/53`

04

The failure ledger

Eighteen entries. The pattern that matters: **almost none of these were catchable by the typechecker, the linter, or the build** — all of which were green throughout — and four of them were the measuring instrument rather than the code.

> A green check is not evidence unless you know what a true positive would look like.

When the instrument lied 4 ENTRIES

F-002

Three "secrets" found in the bundle, none of them secrets

Grepping `dist/` for `sk-ant`, `ANTHROPIC_API_KEY` and `localStorage` hit all three. Every hit was user-facing copy: an input placeholder, an explanatory sentence, and a reassurance that reads "never `localStorage`".

Worse than the false alarms: the same check would have passed trivially earlier in the build, when the AI layer was not in the bundle at all. Replaced with a decoy-*value* build test.

> A word-based grep cannot tell explanatory copy from a leaked value. Ask what a real leak would look like before trusting the absence of one.

F-006

An accessibility audit that invented two defects

The browser tool's accessibility tree reported six unnamed buttons and radios named `exploratory_not_prioritized`. Both bug reports were already being drafted.

Neither existed. The tool surfaces a control's `value` attribute and does not read `sr-only` text. Re-measured against the live DOM: zero unnamed buttons, all radios correctly named.

> Verify a tool's finding before acting on it — especially a negative one. Two invented defects would have produced "fixes" to working code.

F-011

Fourteen contrast failures, one of them real

Four separate measurement traps at once. `background-color` is transparent when a gradient carries the ground, so a naive walker falls through to the page colour. Gradient-*filled* text has `color: transparent` — the gradient is the ink. Elements measured mid-animation are still at `scale(0.98)`, so a 44px target reads 43px. And the browser pane returned torn frames while several agents drove it concurrently.

The one real failure — escalation numerals at **2.32:1** against a 4.5 floor — would have been lost in the noise.

> A measurement harness needs its own verification. Teach it about gradients, wait for animations to settle, and confirm every suspicious frame against the DOM.

F-017

Zero issues reported on visibly broken screens

An automated sweep for clipping and overflow across three tabs at three widths came back clean. A single screenshot from the owner showed "RECIPIENT" spilling its border and role chips stacked one per line as skinny boxes.

All of it was *layout squeeze*, not technical overflow. The chips wrapped legally into a too-narrow column; the label spill sat inside a `min-w-0` container so it never exceeded a scroll box.

> "No measurable overflow" is not "looks right". A layout can be squeezed to uselessness while every box still technically contains its contents. Look first, measure second.

Silent failures with no error anywhere 5 ENTRIES

F-007

The entire type system was inert, and it looked fine

Adding a variable font to the Google Fonts link with five declared axes but only three values per tuple. Google answers a malformed `css2` request with an **HTML error page**, so the whole stylesheet died — taking Inter and JetBrains Mono, which had been working, down with it.

The page still showed a serif headline, because the fallback was a serif. `<link>` failures are silent, and `getComputedStyle` happily reports the font-family you asked for whether or not it exists.

> Found by canvas-measuring the same string in three faces and confirming the widths differed. A screenshot is not proof a font loaded.

F-008

Three cards rendered white because of the alphabet

Three agents independently reported a tinted card rendering plain white. `cn()` was a plain string joiner, so class conflicts fell through to CSS source order — and Tailwind emits colour utilities *alphabetically*. `bg-amber-soft` loses to `bg-surface`; `bg-teal-soft` wins. The winner depended on the colour's name.

Fixed at the cause: `cn()` now de-duplicates background-*colour* utilities, scoped narrowly so `hover:bg-x` never collapses with `bg-y` and gradients still coexist with a colour.

> A bug that depends on a colour's alphabetical position looks like three unrelated one-offs. Two agents patched their own instance with `!important` and moved on; only collecting the reports revealed one cause.

F-009

A one-line utility deleted every card's elevation

A custom `.edge-sheen` class setting `box-shadow: inset …` is emitted after Tailwind's `shadow-*` at equal specificity, so it replaced the whole property rather than adding to it. Every card combining the two rendered completely flat.

Rewritten to compose instead of fight: `box-shadow: inset …, var(--tw-shadow, 0 0 #0000)`. Tailwind initialises that variable on every element, so elevation survives whichever order the classes land in.

> A custom utility that sets a whole shorthand will silently eat the framework's utilities for that property. Compose with the framework's own custom property — one line fixes every call site at once.

F-014

Optional chaining that guarded the wrong thing

Keyboard shortcuts used `target?.matches(…)` to skip the handler while typing. `event.target` is not always an `Element` — with focus on the document it is `window`, which has no `.matches`. The optional chain guards `null`, not a missing method, so the call threw and killed the whole handler.

It worked by luck in manual testing, because real key presses usually target `body`, which does have the method.

> When the uncertainty is the value's *type*, `instanceof` is the guard. An optional chain only reads like a safety check.

F-018

Every result arrived, and every result was thrown away

Repair's analysis never finished. The stage list ran to the last step and stayed there indefinitely — while the network panel showed **both** requests returning 200 with valid, complete JSON.

The liveness guard was cleanup-only: `useEffect(() => () => { alive.current = false }, [])`. React 18 StrictMode mounts, unmounts, then remounts; the first unmount set the flag false and nothing ever set it back. Every completed response after that hit `if (!alive.current) return`.

> This was almost certainly the "last step takes forever" the owner reported — not latency, but a discarded result. A still-loading state and a discarded-result state look identical from outside; check whether the response actually landed before optimising the wait.

Two things that looked like one 3 ENTRIES

F-003

`AI_MODE` did nothing

The documented mode switch had no effect on the client whatsoever. `resolveAiMode()` read only `VITE_AI_MODE`; `AI_MODE` was read exclusively by the Vite plugin, server side. Two variables that looked like one.

Found by the agent writing the README, not by the agent writing the code, and not by the typechecker. A presenter would have hit this live, believing they had switched to fixture mode.

> Documenting a system is a test of it. This is the third defect in this log found while writing docs rather than while writing code.

F-013

A half-configured provider joined the failover chain

"Configured" was defined as key plus base URL. An OpenAI-compatible endpoint without a model joined the chain and sent `model: ''`, so the endpoint rejected it — surfacing as a confusing extra failure instead of the provider simply being absent.

> A half-configured dependency is worse than an absent one: it converts a clear "not available" into a misleading failure.

F-005

Scrolling the page silently reset the demo

A native `<select>` in a sticky header sits under the cursor during ordinary page scrolling, and a focused select changes value on wheel. Mouse-wheeling past it swapped the loaded scenario and reset everything.

> Found by an agent driving the real app with a mouse. Invisible to typecheck, lint, and every unit-level check — and precisely the kind of thing that goes wrong on stage, mid-sentence.

When the code was right and the product was wrong 3 ENTRIES

F-016

The brief was implemented correctly and produced an unusable app

A twelve-feature "Human Observability Console" — five workspaces, a four-region shell, features named Stack Trace, Unit Tests, Patch, Breakpoint, Message Compiler. All gates green. The owner's verdict: *"this ui is terrible, i don't understand what the app is or how to use it."*

Three compounding causes. It opened on an abstract question with six empty panels, each announcing its own emptiness. The twelve names are developer metaphors — without already holding the metaphor, "Stack Trace" says nothing about talking to your wife. And the one thing that sells the product sat four clicks behind a rail of twelve tools.

> A spec can be followed correctly and still produce the wrong product. The brief's own acceptance criterion said a new viewer must understand the premise in under 20 seconds, and its architecture made that impossible. The signal was there and got walked past: when the shell agent reported "six regions, all empty on first paint", **that was the finding**, and it was treated as a status update.

F-004

A comment that documented intent instead of behaviour

Seeding the prepared scenario's follow-up answers made the flow skip the follow-up step entirely — silently deleting the demo beat that shows the product gathering facts rather than guessing. The code comment asserted "the presenter still sees it." It did not.

Fixed by keeping the seeded answers but not marking the step resolved, so the questions render with every choice pre-selected. Nothing is asked; nothing is invented; the beat survives.

> "Don't ask" and "don't show" are different requirements. A comment that disagrees with its code is worse than no comment — it stops the next reader from looking.

F-012

A cosmetic change broke an honesty guarantee

Adding provider names to the status pill so "Live" says which provider served the result. The bring-your-own-key path — the user's key, sent from their browser — started rendering identically to the server route, erasing the one privacy-relevant distinction the indicator exists to make.

> An honesty guarantee can be broken by a change that looks purely presentational. Scoped back to the proxy source only, with a comment saying why, so it does not get tidied up later.

Also logged

- **F-001** — `z.discriminatedUnion` rejects a schema wrapped by `.superRefine()`, because `ZodEffects` no longer exposes the discriminator. The refined schema stays the real gate; the union gets `.innerType()`.
- **F-010** — Every live call failed 502 with an opaque client error. The provider's own message read "credit balance is too low". The proxy deliberately never forwarded provider text, which also hid the one line that explained the failure. Now forwarded for configuration-class statuses only. **A billing rejection is diagnostically valuable: a 400 rather than a 401 proves auth, request shape, and transport all work.**
- **F-015** — The dark theme put `text-surface` on a gradient. Correct in eight light themes where `surface` is white; 4.09:1 in a dark theme where it is near-black. Any component naming a specific colour for foreground-on-accent carries an assumption about the palette's polarity.

05

What verification actually proved

Two automated gates, plus measurement in the running app. The distinction between *verified* and *assumed* is kept explicit — several rows below are deliberately recorded as partial.

CHECK	RESULT	HOW
Fixture gate	60 / 60	Schema plus content invariants. Negative-tested: injecting "the sender is annoyed" into the decode fixture turns the build red.

CHECK	RESULT	HOW
Functional gate	`53 / 53`	State machine, request building, validation. Asserts the role pair and follow-up answers actually reach the request payload.
No key in the bundle	`Pass`	Decoy-value production build, then grep `dist/` for the decoy. No matches.
Provider failover	`Pass`	Proven end to end against a local mock, then observed in production when Anthropic ran out of credit.
Text contrast	`0 failures`	443 text nodes across five screens, computed from rendered colours with gradient-aware resolution.
Tap targets	`0 under 44px`	Measured at 375 / 768 / 1440 after animations settle.
Repair time to first content	`2.1s`	Sixteen-message conversation, OpenAI first. Full analysis at 8.6s behind it.
Keyboard traversal	`Partial`	Every control is native with an accessible name (0 unnamed, verified live). A full end-to-end tab pass was not redone after the redesign — recorded as untested rather than assumed.
Reduced motion	`Structural`	One global media block; 14 of 14 animations additionally behind `motion-safe:`. Not verified by toggling the OS setting — the harness exposes no way to emulate it.
Copy confirmation	`Partial`	The failure path was observed end to end. The success path needs genuine user activation, which synthesised clicks did not reliably grant. A presenter should click Copy once by hand first.

06

The last mile

Three changes shipped after the product was otherwise finished, all of them about the demo surviving contact with the room.

Progressive loading

Repair's full analysis is the largest response the app produces. A second, deliberately small request now runs *in parallel* — capped at 500 tokens, asking only what each side seems to be saying and what it is probably about — and renders as soon as it lands. It is best-effort by construction: it never throws, never blocks, and returns `null` on any failure, so a slow first read leaves the normal wait exactly as it was. Measured: first content at 2.1s instead of 8.6s.

Four decisions collapsed into three screens

Repair used to ask the user to upload, wait, accept a transcript, and then request the analysis. Three of those four asked the user to approve the app's own progress. Now: put it in, check it, read it. Text written in someone's own words skips the check screen entirely — there is nothing to proofread. A pasted transcript still gets it, because pasted text carries the same speaker-labelling risk a screenshot does.

A fallback that is a cache, not a fake

If no provider answers during the demo, Repair serves a prepared analysis of the exact conversation that was submitted, behind a banner saying it is a saved read. This does not bend the product's honesty rule, and the distinction is the whole point: substituting unrelated fixture content for a user's text would be a fake; serving a prepared analysis *of the same conversation* is a cache.

The matcher is strict for exactly that reason — six of ten distinctive lines, gate-verified to survive OCR dropping messages, and to **refuse** both the other conflict fixture and a differently-worded argument about the same dinner. Speaker ids bind at runtime, because a screenshot of that conversation contains no names at all, and the builder returns `null` rather than guess when it cannot tell who is who.

07

Known limits

- **The Anthropic account has no credit.** It returns 400 on every call, so OpenAI now leads the chain and the demo runs on `gpt-4o`. The failover works correctly; the reorder just avoids paying for a failed attempt on every request.
- **Jargon control is a three-way UI over a boolean contract.** "Reduce" and "remove" currently behave identically at the prompt level — the interface offers a distinction the model does not receive. Flagged rather than hidden; it needs a real contract field.
- **Nine themes is more than a demo can explain.** All nine are contrast-audited, but two or three would be clearer.

- **Multi-speaker conflicts are out of scope.** More than two speakers is reported, not resolved — the app says so rather than guessing a mapping.

```
Context Switch · build record compiled from HACKATHON_BUILD_LOG.md
9 commits · 20 decisions · 18 failures · 113 automated checks
Every figure here was measured. Where something could not be verified, it says so.
```